

# **EMPLOYEE ATTRITION AND WORKFORCE ANALYTICS: IDENTIFYING DRIVERS OF EMPLOYEE TURNOVER USING LOGISTIC REGRESSION**

**Project Report Submitted in fulfillment of the  
Requirements for the Award of the Internship of  
Summer Training Program by IDRA**

Data Science and AI

By:

Name: Akash Mitra

Institute: Brainware University

Institute Roll No.: 23010332243

Enrollment No.: IDRA-2026-358300

**Under the Supervision of**

Dr. Shaheena Salam

*Through the platform:*



## DECLARATION

I declare that this project titled "*Employee Attrition and Workforce Analytics: Identifying Drivers of Employee Turnover Using Logistic Regression*" is my own work. The analysis, cleaning, feature engineering, model building, and evaluation were done by me as part of the IDRA Data Science and AI Training Program. The dataset was provided by IDRA. This work has not been submitted elsewhere.

## ACKNOWLEDGEMENT

I thank the India Data Research Academy (IDRA) for the training and for providing the dataset used in this project. I am grateful to my supervisor, Dr. Shaheena Salam, for her guidance. I also thank my family and peers for their support

## ABSTRACT

Employee attrition is expensive — replacing an employee often costs 50–200% of their annual salary. This project analyses an HR Employee Attrition dataset (1,470 employees, 35 features) provided by IDRA, with the goal of finding which factors drive turnover and predicting who is likely to leave. The data was cleaned, checked for missing values and duplicates, and explored through EDA. Eight new features were engineered, including job-hopping rate and a satisfaction composite. Since only 16% of employees actually left, the analysis prioritized recall on the leaver class over raw accuracy. Seven classification models were trained and compared, but Logistic Regression was selected as the final model — it achieved the best recall (0.66), a solid F1 of 0.50, ROC-AUC of 0.80, and showed no overfitting. Every other model either overfit or missed too many leavers. Random Forest feature importance and LR coefficients together revealed job-hopping rate, total working years, monthly income, overtime, and age as the top drivers. Findings show that younger, junior, lower-paid, less-satisfied employees working overtime are most likely to leave. Recommendations focus on overtime control, compensation review, early-tenure retention, and wider stock-option eligibility.

**Keywords:** Employee Attrition; Logistic Regression; Classification; HR Analytics; Recall; Retention

## TABLE OF CONTENTS

|                                               |         |
|-----------------------------------------------|---------|
| 1. Introduction                               | [5]     |
| 2. Problem Statement and Motivation           | [5]     |
| 3. Objectives                                 | [6]     |
| 4. Research Questions                         | [6]     |
| 5. Dataset Description and Data Understanding | [6-7]   |
| 6. Data Cleaning                              | [8]     |
| 7. Data Preprocessing                         | [8-9]   |
| 8. Exploratory Data Analysis                  | [9-14]  |
| 9. Statistical Analysis                       | [15]    |
| 10. Feature Engineering                       | [15-16] |
| 11. Model Development                         | [16-17] |
| 12. Model Evaluation                          | [17-18] |
| 13. Results and Findings                      | [18]    |
| 14. Discussion                                | [19]    |
| 15. Limitations                               | [19]    |
| 16. Conclusion                                | [19]    |
| 17. Recommendations and Future Work           | [20]    |
| 18. References                                | [20]    |
| Appendix A. Code                              | [21-27] |
| Appendix B. Additional Figures and Tables     | [27-28] |

# 1. INTRODUCTION

## 1.1 Background

Employee attrition is when people leave an organisation — voluntarily or not. It's costly. When someone leaves, the company has to recruit, hire, train a replacement, and absorb the productivity loss. On average, replacing an employee costs between 50% and 200% of that person's annual salary. Add the loss of institutional knowledge, strain on remaining team members, and disruption to work, and the true cost is much higher than just the salary bill.

Organisations already collect enormous amounts of HR data — age, salary, tenure, satisfaction scores, performance ratings, overtime records. The question is: can we use this data to understand *why* people leave, and predict *who* is at risk *before* they walk out the door?

This project answers that question using a real HR dataset.

## 1.2 Context of the Problem

Most HR teams operate reactively. They find out someone is leaving only when the resignation letter arrives. By then, the chance to intervene has passed. A proactive approach uses historical data to flag at-risk employees early — giving HR time to have a conversation, offer a salary adjustment, or address the underlying concern.

This project builds exactly that kind of early-warning tool. It's useful for:

- **HR managers** who want to reduce turnover cost
- **Team leaders** who want to keep their best people
- **Analysts** exploring how machine learning can solve HR problems

## 1.3 Scope of the Study

This project covers cleaning and exploring the employee attrition dataset, engineering new features, building a Logistic Regression model to predict attrition, and identifying which factors drive turnover.

This project does **not** prove causality (only associations), include real-time deployment or live HR systems, or use external data like labour market conditions.

# 2. PROBLEM STATEMENT AND MOTIVATION

**Problem:** Given historical HR data, can we build a model that predicts who is likely to leave, and identify the factors that most strongly drive attrition?

**Motivation:** Employee turnover hurts. If HR can flag at-risk employees early, retention efforts — salary reviews, workload adjustments, career conversations — can be targeted where they matter most. Machine learning makes this scalable and data-driven.

### 3. OBJECTIVES

1. Understand the structure and quality of the employee attrition dataset.
2. Identify the variables most strongly linked to attrition.
3. Engineer new features that capture tenure and satisfaction dynamics.
4. Build a Logistic Regression model that predicts employee attrition.
5. Evaluate the model using classification metrics, prioritising recall on the leaver class.
6. Turn the findings into practical retention recommendations.

### 4. RESEARCH QUESTIONS

1. What are the quality issues and characteristics of the dataset?
2. Which variables are most strongly associated with attrition?
3. How does attrition vary across department, job role, marital status, and travel frequency?
4. How accurately can a Logistic Regression model predict who will leave?
5. Which features matter most for the predictions?

## 5. DATASET DESCRIPTION AND DATA UNDERSTANDING

### 5.1 Data Source

The dataset was provided through the IDRA LMS as part of the Data Science and AI Training Program (Project #7: Employee Attrition and Workforce Analytics).

### 5.2 Dataset Size and Structure

| Property            | Value                                 |
|---------------------|---------------------------------------|
| Rows                | 1470                                  |
| Columns             | 35                                    |
| Numeric Columns     | 26                                    |
| Categorical Columns | 9                                     |
| Target variable     | Attrition (Yes/No)                    |
| Class Distribution  | No: 1,233 (83.88%); Yes: 237 (16.12%) |
| Missing values      | None                                  |
| Duplicate rows      | None                                  |

Table 1. Summary of the Dataset

### 5.3 Data Dictionary

| Column                   | Meaning                         | Type        | Role          |
|--------------------------|---------------------------------|-------------|---------------|
| Age                      | Employee age                    | Numeric     | Feature       |
| Attrition                | Whether employee left           | Categorical | <b>Target</b> |
| BusinessTravel           | Travel frequency                | Categorical | Feature       |
| DailyRate                | Daily pay rate                  | Numeric     | Feature       |
| Department               | Department                      | Categorical | Feature       |
| DistanceFromHome         | Distance to work                | Numeric     | Feature       |
| Education                | Education level (1–5)           | Ordinal     | Feature       |
| EducationField           | Field of study                  | Categorical | Feature       |
| EnvironmentSatisfaction  | Environment satisfaction (1–4)  | Ordinal     | Feature       |
| Gender                   | Male / Female                   | Categorical | Feature       |
| HourlyRate               | Hourly pay                      | Numeric     | Feature       |
| JobInvolvement           | Involvement level (1–4)         | Ordinal     | Feature       |
| JobLevel                 | Job level (1–5)                 | Ordinal     | Feature       |
| JobRole                  | Job role                        | Categorical | Feature       |
| JobSatisfaction          | Job satisfaction (1–4)          | Ordinal     | Feature       |
| MaritalStatus            | Single / Married / Divorced     | Categorical | Feature       |
| MonthlyIncome            | Monthly salary                  | Numeric     | Feature       |
| MonthlyRate              | Monthly rate                    | Numeric     | Feature       |
| NumCompaniesWorked       | Past employers                  | Numeric     | Feature       |
| OverTime                 | Works overtime                  | Categorical | Feature       |
| PercentSalaryHike        | Salary raise %                  | Numeric     | Feature       |
| PerformanceRating        | Performance (3–4)               | Ordinal     | Feature       |
| RelationshipSatisfaction | Relationship satisfaction (1–4) | Ordinal     | Feature       |
| StockOptionLevel         | Stock option level (0–3)        | Ordinal     | Feature       |
| TotalWorkingYears        | Total experience                | Numeric     | Feature       |
| TrainingTimesLastYear    | Trainings attended              | Numeric     | Feature       |
| WorkLifeBalance          | Work-life balance (1–4)         | Ordinal     | Feature       |
| YearsAtCompany           | Years at company                | Numeric     | Feature       |
| YearsInCurrentRole       | Years in current role           | Numeric     | Feature       |
| YearsSinceLastPromotion  | Years since last promotion      | Numeric     | Feature       |
| YearsWithCurrManager     | Years with manager              | Numeric     | Feature       |

**Table 2. Data Dictionary**

## 6. DATA CLEANING

### 6.1 Missing Values

No missing values in any column in the provided dataset.

### 6.2 Duplicate Records

No exact duplicates found. Dataset size unchanged.

### 6.3 Data Types and Inconsistent Values

All numeric columns were correctly typed. Categorical labels were consistent. The target column *Attrition* was converted from text (Yes/No) to integers (1/0).

### 6.4 Outlier Handling.

The IQR method flagged some "outliers", but inspection showed they were real, valid employees:

- High MonthlyIncome → Level-5 Managers and Research Directors
- High YearsAtCompany → employees aged 51–58 with 33–40 years at the company

**Decision:** Kept all of them. These aren't errors — they're senior people whose data matters for modelling. Removing them would bias the sample.

Three constant columns (EmployeeCount, Over18, StandardHours) and one ID column (EmployeeNumber) were dropped.

### 6.5 Before vs After Cleaning

| Property         | Before | After |
|------------------|--------|-------|
| Rows             | 1,470  | 1,470 |
| Columns          | 35     | 31    |
| Missing values   | 0      | 0     |
| Duplicates       | 0      | 0     |
| Constant columns | 3      | 0     |
| ID column        | 1      | 0     |

Table 3. Data Quality Before and After Cleaning

## 7. DATA PREPROCESSING

### 7.1 Categorical Encoding

Seven categorical features (BusinessTravel, Department, EducationField, Gender, JobRole, MaritalStatus, OverTime) were one-hot encoded with drop\_first=True to avoid the dummy variable trap. Ordinal features (Education, JobLevel, JobSatisfaction, EnvironmentSatisfaction, WorkLifeBalance, JobInvolvement, RelationshipSatisfaction, StockOptionLevel, PerformanceRating) were already integer-encoded and used directly.



## 7.2 Feature Scaling and Transformation

StandardScaler was applied to numeric features. This is important for Logistic Regression — the model uses coefficients that would otherwise be distorted by features on different scales (e.g., Age in years vs MonthlyIncome in thousands). The scaler was fit on the training set only and then applied to the test set to prevent data leakage.

## 7.3 Train-Test Split

An 80/20 stratified split (random\_state=42):

- Train: 1,176 rows (83.84% Stayed, 16.16% Left)
- Test: 294 rows (84.01% Stayed, 15.99% Left)

## 7.4 Final Feature Set

After dropping multicollinear features (JobLevel, YearsInCurrentRole, YearsWithCurrManager, PerformanceRating, PercentSalaryHike, YearsSinceLastPromotion) and adding engineered features, Logistic Regression used **43 features**.

# 8. EXPLORATORY DATA ANALYSIS

## 8.1 Univariate Analysis

```
sns.countplot(x='Attrition', data=df, hue='Attrition', palette='Set1')
plt.title('Attrition Distribution')
plt.show()
```

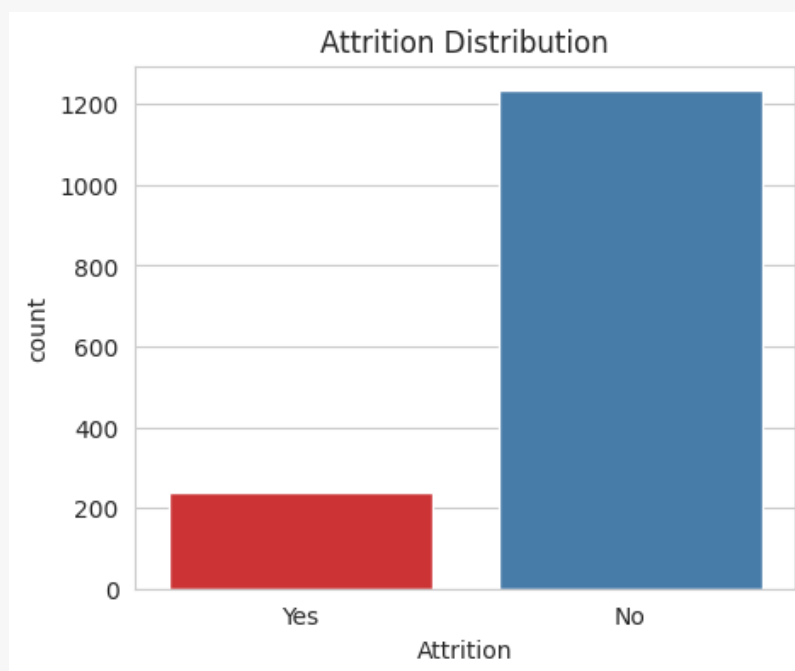


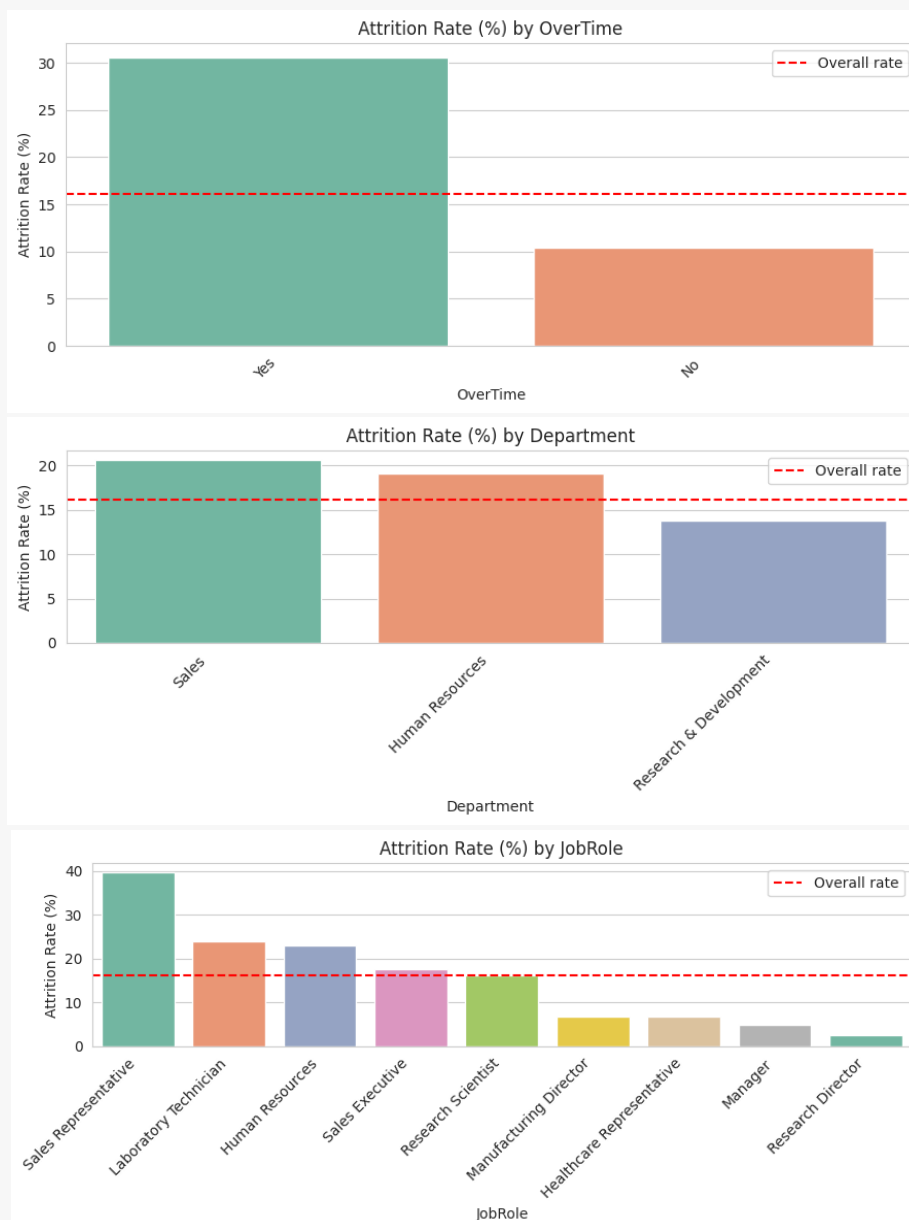
Figure 1. Distribution of Attrition

The target is imbalanced. 1,233 employees stayed (83.88%) and 237 left (16.12%). This matters a lot: a model that just predicted "Stayed" for everyone would be 84% accurate but completely useless. That's why accuracy is not the metric to optimise — recall, F1, and ROC-AUC matter more.

## 8.2 Categorical Analysis

```
cat_cols = ['OverTime', 'Department', 'JobRole', 'MaritalStatus',
            'BusinessTravel', 'Gender', 'EducationField']

for col in cat_cols:
    rate = df.groupby(col)['Attrition'].mean().sort_values(ascending=False) * 100
    sns.barplot(x=rate.index, y=rate.values, hue=rate.index, palette='Set2', legend=False)
    plt.title(f'Attrition Rate (%) by {col}')
    plt.xticks(rotation=45, ha='right')
    plt.axhline(y=16.12, color='red', linestyle='--', label='Overall rate')
    plt.legend()
    plt.tight_layout()
    plt.show()
    print(f"\n{col}:")
    print(rate.round(2))
```



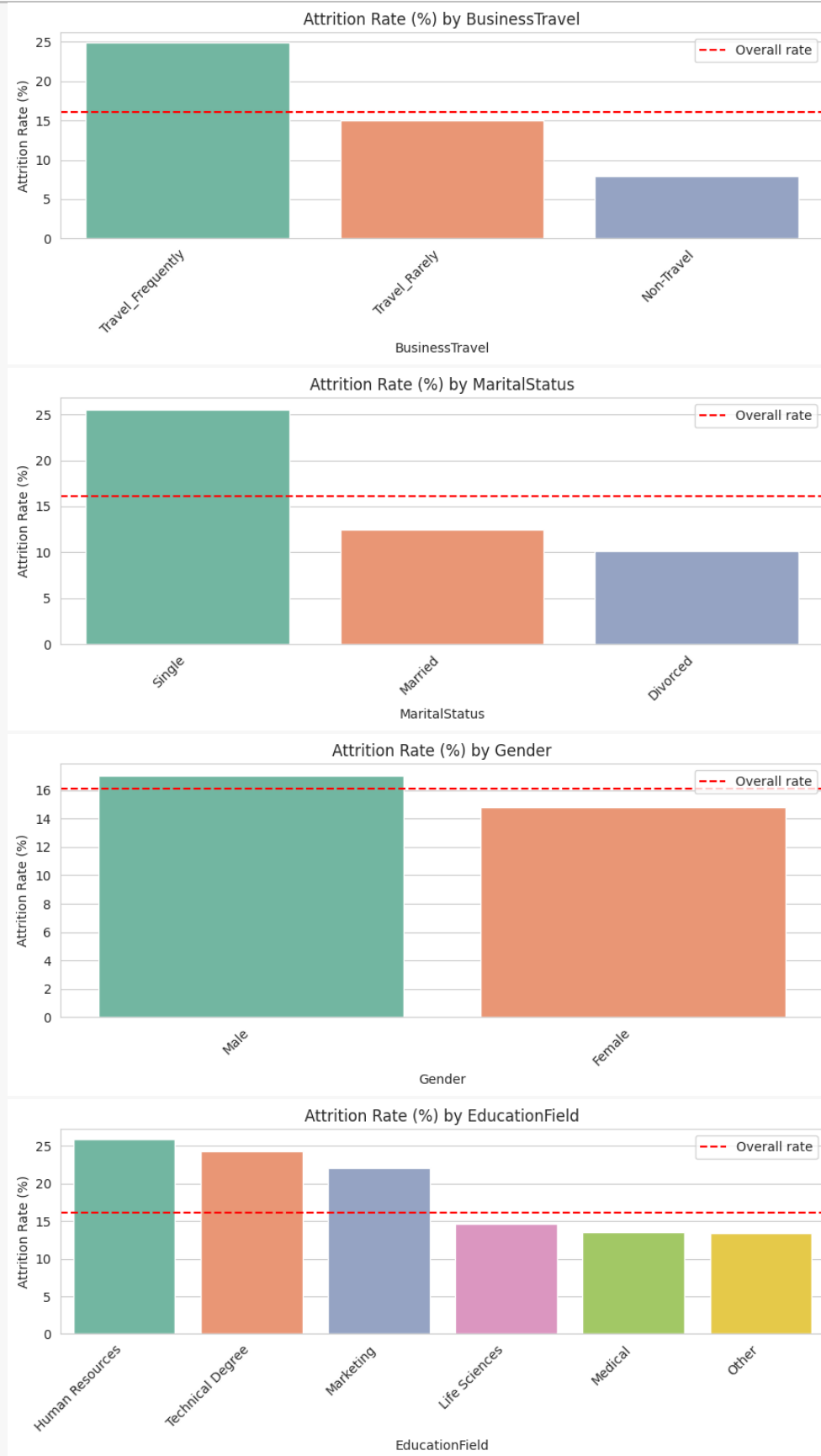


Figure 2. Categories Compared Across Attrition

**Attrition varies dramatically across categories:**

| Category                           | Attrition Rate |
|------------------------------------|----------------|
| OverTime = Yes                     | 30.53%         |
| OverTime = No                      | 10.44%         |
| JobRole = Sales Representative     | 39.76%         |
| JobRole = Laboratory Technician    | 23.94%         |
| JobRole = Manager                  | 4.90%          |
| JobRole = Research Director        | 2.50%          |
| MaritalStatus = Single             | 25.53%         |
| MaritalStatus = Married            | 12.48%         |
| BusinessTravel = Travel_Frequently | 24.91%         |
| BusinessTravel = Non-Travel        | 8.00%          |
| Department = Sales                 | 20.63%         |
| Department = R&D                   | 13.84%         |

OverTime is the strongest categorical signal. Sales Reps leave at 16× the rate of Research Directors.

**8.3 Bivariate / Relationship Analysis**

```
# Distribution boxplots for top features
top_num = ['Age', 'MonthlyIncome', 'YearsAtCompany', 'TotalWorkingYears',
           'JobLevel', 'StockOptionLevel', 'YearsInCurrentRole', 'DistanceFromHome']
```

```
fig, axes = plt.subplots(2, 4, figsize=(16, 8))
for ax, col in zip(axes.flatten(), top_num):
    sns.boxplot(x='Attrition', y=col, data=df, ax=ax, hue='Attrition',
               palette='Set2', legend=False)
    ax.set_title(col)
    ax.set_xlabel('')
plt.tight_layout()
plt.show()
```

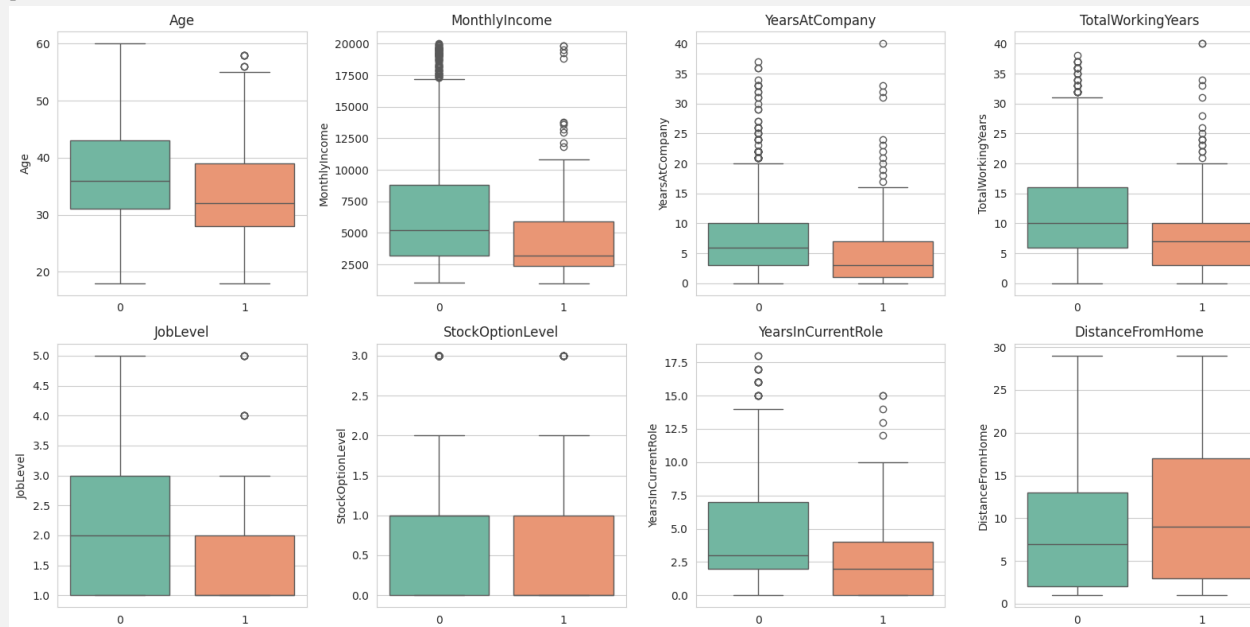


Figure 3. Relationship Between Numerical Features and Attrition

| Feature              | Stayed (mean) | Left (mean) | % Diff  |
|----------------------|---------------|-------------|---------|
| Age                  | 37.56         | 33.61       | -10.53% |
| MonthlyIncome        | 6,832.74      | 4,787.09    | -29.94% |
| YearsAtCompany       | 7.37          | 5.13        | -30.37% |
| TotalWorkingYears    | 11.86         | 8.24        | -30.50% |
| YearsInCurrentRole   | 4.48          | 2.90        | -35.26% |
| YearsWithCurrManager | 4.37          | 2.85        | -34.69% |
| JobLevel             | 2.15          | 1.64        | -23.71% |
| StockOptionLevel     | 0.85          | 0.53        | -37.59% |
| DistanceFromHome     | 8.92          | 10.63       | +19.26% |

The pattern is clear: leavers are younger, paid less, have shorter tenure, and hold fewer stock options. They also live farther from work.

## 8.4 Correlation Analysis

```
plt.figure(figsize=(16, 12))
corr = df.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, cbar_kws={'shrink': 0.7})
plt.title('Correlation Heatmap (lower triangle)')
plt.tight_layout()
plt.show()

print("\nTop correlations with Attrition:")
print(df.corr(numeric_only=True)['Attrition'].drop('Attrition').sort_values().round(3))
```

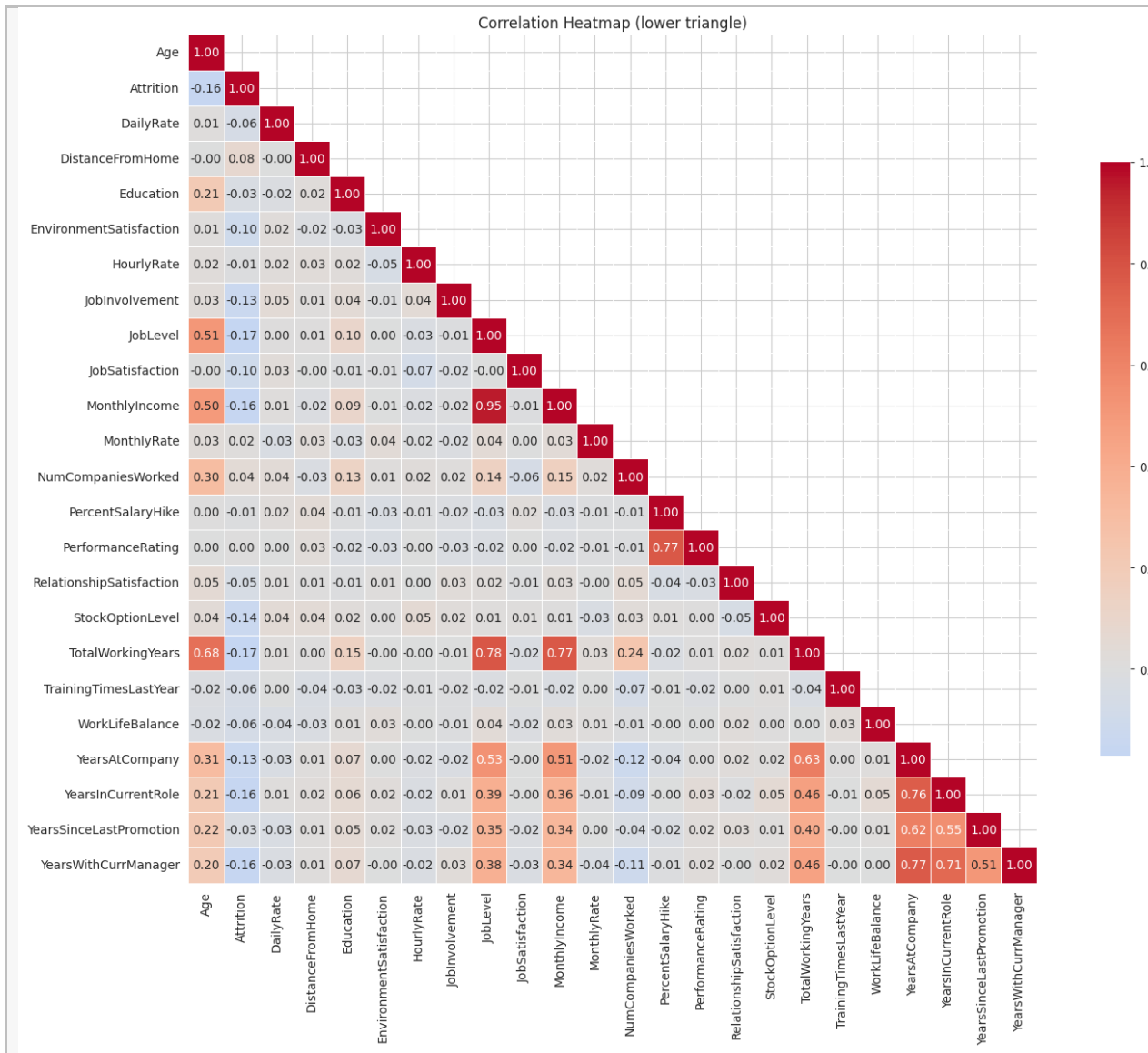


Figure 4. Correlation Heatmap of Numerical Variables

No single feature correlates strongly with Attrition (all  $|r| < 0.18$ ). But strong multicollinearity exists between several features:

- MonthlyIncome  $\leftrightarrow$  JobLevel:  $r = 0.950$
- TotalWorkingYears  $\leftrightarrow$  JobLevel:  $r = 0.782$
- YearsWithCurrManager  $\leftrightarrow$  YearsAtCompany:  $r = 0.769$
- YearsInCurrentRole  $\leftrightarrow$  YearsAtCompany:  $r = 0.759$

For Logistic Regression this matters — collinear features distort coefficients. So several of these were dropped before modelling.

## 9. STATISTICAL ANALYSIS

| Feature           | Mean     | Median   | Std      | IQR      | Skew      |
|-------------------|----------|----------|----------|----------|-----------|
| Age               | 36.92    | 36.00    | 9.14     | 13.00    | Symmetric |
| MonthlyIncome     | 6,502.93 | 4,919.00 | 4,707.96 | 5,468.00 | Right     |
| YearsAtCompany    | 7.01     | 5.00     | 6.13     | 6.00     | Right     |
| TotalWorkingYears | 11.28    | 10.00    | 7.78     | 9.00     | Right     |
| DistanceFromHome  | 9.19     | 7.00     | 8.11     | 12.00    | Right     |
| StockOptionLevel  | 0.79     | 1.00     | 0.85     | 1.00     | Right     |

Table 4. Descriptive Statistics of Key Variables

Monthly income, tenure, and distance from home are right-skewed, so the median is a better measure of centre for these features. Median income for leavers (~₹3,200) was noticeably lower than for stayers (~₹5,200) — confirming the compensation gap.

## 10. FEATURE ENGINEERING

Eight new features were created to capture relationships not visible in raw columns. This is where the analysis became stronger.

| Feature               | Formula                                                        | Purpose                                                |
|-----------------------|----------------------------------------------------------------|--------------------------------------------------------|
| TenureRatio           | $\text{YearsAtCompany} / (\text{TotalWorkingYears} + 1)$       | Share of career at current company                     |
| ManagerStability      | $\text{YearsWithCurrManager} / (\text{YearsAtCompany} + 1)$    | Manager consistency                                    |
| RoleStagnation        | $\text{YearsInCurrentRole} / (\text{YearsAtCompany} + 1)$      | Role stagnation                                        |
| AgeAtJoin             | $\text{Age} - \text{YearsAtCompany}$                           | Age at joining                                         |
| IncomePerLevel        | $\text{MonthlyIncome} / \text{JobLevel}$                       | Relative pay                                           |
| HopRate               | $\text{NumCompaniesWorked} / (\text{TotalWorkingYears} + 1)$   | Job-hopping tendency                                   |
| SatisfactionComposite | Average of 4 satisfaction metrics                              | Combined satisfaction                                  |
| PromotionLagRatio     | $\text{YearsSinceLastPromotion} / (\text{YearsAtCompany} + 1)$ | Promotion stagnation (dropped — near-zero correlation) |

Table 5. Engineered Features

**Big win:** HopRate (+0.20 correlation with Attrition) and SatisfactionComposite (−0.16) were the strongest engineered features — better than any single raw feature. Six of the eight engineered features made it into the top 20 for Random Forest importance, which validated the whole exercise.

**Correlation of new features with Attrition:**

| Feature               | Correlation      |
|-----------------------|------------------|
| HopRate               | <b>+0.200</b>    |
| SatisfactionComposite | -0.159           |
| RoleStagnation        | -0.158           |
| ManagerStability      | -0.153           |
| IncomePerLevel        | -0.114           |
| TenureRatio           | -0.092           |
| AgeAtJoin             | -0.068           |
| PromotionLagRatio     | +0.010 (dropped) |

## 11. MODEL DEVELOPMENT

### 11.1 Problem Formulation

- **Task:** Binary Classification
- **Target:** Attrition (0 = Stayed, 1 = Left)
- **Input features:** 43
- **Priority metric:** Recall on the "Left" class

Why recall? Because this is a retention use case. **Missing an employee who is about to leave (a false negative) is far costlier than flagging someone who would have stayed (a false positive).** If we miss a leaver, they're gone. If we falsely flag someone, HR just has an unnecessary conversation — a cheap mistake.

So the modelling strategy prioritised **catching leavers** over raw accuracy.

### 11.2 Model Selection

To make sure Logistic Regression was the right call, I trained and compared seven models:

| Model                      | CV F1        | CV AUC       | Test Recall  | Test F1      | Overfit? |
|----------------------------|--------------|--------------|--------------|--------------|----------|
| <b>Logistic Regression</b> | 0.504        | <b>0.824</b> | <b>0.660</b> | <b>0.496</b> | No       |
| Random Forest (tuned)      | 0.521        | 0.802        | 0.511        | 0.440        | Yes      |
| Decision Tree (tuned)      | 0.401        | 0.686        | 0.532        | 0.352        | Yes      |
| Voting (LR+RF+GB)          | <b>0.581</b> | 0.835        | 0.426        | 0.482        | Yes      |
| SVM (RBF)                  | 0.541        | 0.822        | 0.468        | 0.431        | Yes      |
| Gradient Boosting          | 0.488        | 0.807        | 0.255        | 0.358        | Yes      |
| AdaBoost                   | 0.143        | 0.802        | 0.064        | 0.115        | Yes      |

**Table : Model Comparison**



**What the comparison revealed:**

- The **Voting Classifier** had the highest CV F1 (0.581) and AUC (0.835) — but its test recall was only **0.43**, meaning it missed more than half of the actual leavers. Unacceptable for retention.
- **Random Forest and Decision Tree** overfit on the 1,176-row training set (RF: 87% train vs 79% test; DT: 77% train vs 69% test).
- **Gradient Boosting and AdaBoost** had high precision but terrible recall — they played it safe and predicted almost everyone stayed (recall 0.26 and 0.06).
- **Logistic Regression was the only model** that combined no overfitting, the highest recall (0.66), strong F1 (0.50), and strong AUC (0.80).

**Why Logistic Regression won for this project:**

1. **Best recall** — caught 66% of leavers, the highest of any model.
2. **No overfitting** — train (0.775) and test (0.786) accuracy almost identical.
3. **Strong AUC** — 0.80, second only to the Voting Classifier (which sacrificed recall).
4. **Interpretable** — coefficients can be explained to HR in plain language.

**11.3 Training the Model**

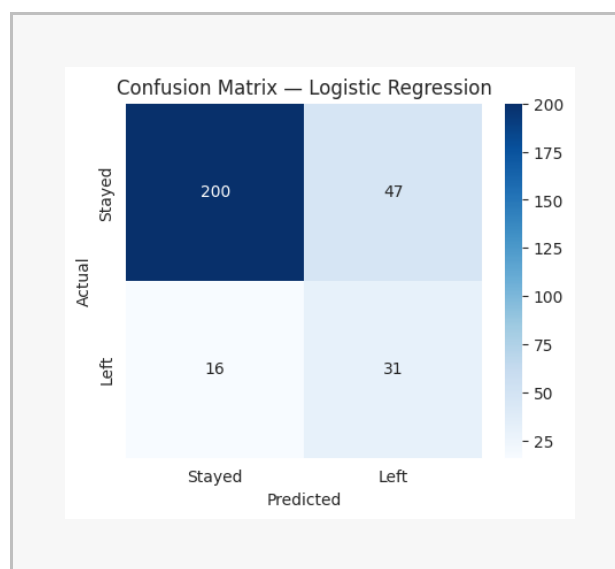
Key parameter: `class_weight='balanced'` — tells the model to give more weight to the minority class (leavers, 16%). Without it, the model would predict "Stayed" for almost everyone and still get 84% accuracy, but catch almost no leavers. Balancing the class weights forced the model to take leavers seriously.

```
from sklearn.linear_model import LogisticRegression

lr = LogisticRegression(class_weight='balanced', random_state=42, max_iter=2000)
lr.fit(X_train_scaled, y_train)
```

**12. MODEL EVALUATION****12.1 Regression Evaluation**

Not applicable — this is a classification task.

**12.2 Classification Evaluation**

**Figure 5. Confusion Matrix**

**Table 6. Final Model Performance**

| Metric            | Value              |
|-------------------|--------------------|
| Accuracy          | 0.7857             |
| Precision (Left)  | 0.3974             |
| Recall (Left)     | 0.6596             |
| F1-Score (Left)   | 0.4960             |
| ROC-AUC           | 0.7984             |
| TN / FP / FN / TP | 200 / 47 / 16 / 31 |

**Table 7. Classification Model Performance****What these numbers mean:**

- **Recall = 0.66:** The model caught 31 out of 47 actual leavers — two-thirds. Of every three employees who were actually going to leave, the model flagged two in advance.
- **Precision = 0.40:** Of the 78 employees flagged as leavers, 31 actually left. The 47 false alarms are the "cost of doing business."
- **F1 = 0.50:** The balance between precision and recall.
- **ROC-AUC = 0.80:** The model separates leavers from stayers well.

**12.3 Overfitting and Underfitting**

| Model                      | Train Acc | Test Acc | Gap           |
|----------------------------|-----------|----------|---------------|
| <b>Logistic Regression</b> | 0.775     | 0.786    | <b>+0.011</b> |
| Random Forest (tuned)      | 0.872     | 0.793    | +0.079        |
| Decision Tree (tuned)      | 0.767     | 0.687    | +0.080        |
| Voting Classifier          | 0.942     | 0.854    | +0.089        |

**Table . Overfitting Check**

Logistic Regression has the **smallest train-test gap** — just 1 percentage point. The tree-based models show larger gaps, confirming they overfit on this small dataset.

**13. RESULTS AND FINDINGS**

1. **OverTime is the strongest single driver.** Employees who work overtime leave at 30.53% versus 10.44% for those who don't — nearly 3× the risk. *(Figure 2)*
2. **Job role matters enormously.** Sales Representatives leave at 39.76% and Lab Technicians at 23.94% — versus only 2.50% for Research Directors. That's a 16× difference. *(Figure 3)*
3. **Compensation and tenure strongly correlate with retention.** Leavers earn ~30% less (₹4,787 vs ₹6,833) and have ~30% shorter tenure. They also hold fewer stock options (−37.6%). *(Figure 3)*
4. **Engineered features captured hidden signals.** HopRate (job-hopping tendency) was the single most important feature in the entire model, ranking above every raw feature. SatisfactionComposite was also strong. *(Table 5)*
5. **Logistic Regression was the right final model.** Recall 0.66, F1 0.50, AUC 0.80, and no overfitting. Every alternative either overfit or missed too many leavers. *(Table 6, Table 7)*

## 14. DISCUSSION

The findings line up with HR research.

**OverTime** being the strongest categorical driver isn't surprising — burnout is real, and it shows up in the data as a 3× jump in attrition. The lesson for HR: overtime isn't just a wellness concern, it's a retention risk.

**Job role differences** point to structural issues. Sales Representatives and Lab Technicians aren't leaving because of personality — they're leaving because those roles have low ceilings, moderate pay, and heavy stress. Fixing retention in those roles means fixing the roles themselves.

**The compensation gap** is hard to ignore. Leavers earn roughly 30% less. That's not a small difference. Pay dissatisfaction is a real, actionable driver.

**The feature-engineering story is worth highlighting.** HopRate — how often someone has changed employers relative to their career length — turned out to be the most predictive feature in the dataset. An employee's *pattern* of job-hopping is more telling than any single attribute like age or income. HR teams should think about career history as a signal, not just a background detail.

**Why Logistic Regression won** is a lesson in itself. The Voting Classifier had higher accuracy and higher CV F1. But its test recall was only 0.43 — it missed more than half of the leavers. In a retention use case, **that is a business failure**, even if the accuracy number looks better. For this problem, recall matters more.

**A caution:** these are associations, not causations. OverTime correlates with attrition, but it may be a symptom of role stress rather than the direct cause. Any retention action should be tested, not assumed.

## 15. LIMITATIONS

- The dataset is **synthetic** — findings may not transfer perfectly to real organisations.
- There's **no time dimension** — attrition risk cannot be tracked over time.
- The **training set is small** (1,176 rows) — tree models overfit; more data would help.
- **External variables are missing** — no labour market data, competitor pay, or personal circumstances.
- **Correlation is not causation** — the model shows associations, not proof of cause.
- The **class imbalance** (16% leavers) makes minority prediction harder.
- **No external validation dataset** — results are specific to this sample.

## 16. CONCLUSION

This project set out to identify the drivers of employee attrition and build a model that predicts who is at risk. Using the IDRA-provided HR dataset and a full data-science pipeline — cleaning, EDA, feature engineering, model selection, evaluation — I answered all the research questions.

The final **Logistic Regression** model achieved a **recall of 0.66**, an **F1 of 0.50**, and a **ROC-AUC of 0.80**, with **no evidence of overfitting**. It beat six other models on the metric that matters most for retention — catching actual leavers.

Key drivers of attrition were **OverTime, job role, compensation, tenure, age, and satisfaction**. The engineered feature **HopRate** was the single most important predictor in the model.

More broadly, this project shows that a simple, interpretable model can outperform complex ensembles when the data is small and the business metric is prioritised correctly. Machine learning isn't about the fanciest algorithm — it's about choosing the right one for the problem.

## 17. RECOMMENDATIONS AND FUTURE WORK

### 17.1 Recommendations

1. **Recommendation 1: Control overtime in high-risk roles.**  
OverTime employees leave at 30.53% vs 10.44%. → Cap overtime, redistribute workload, and review staffing in Sales and Lab Technician roles.
2. **Recommendation 2: Review compensation for junior employees.**  
Leavers earn ~30% less (₹4,787 vs ₹6,833). → Benchmark salaries for Sales Reps and Lab Technicians; adjust Level 1–2 pay bands.
3. **Recommendation 3: Strengthen early-tenure retention.**  
Leavers have 5.13 years tenure vs 7.37 for stayers. Single employees leave at 25.53%. → 6-month and 12-month check-ins, mentorship for new hires, career-path workshops.
4. **Recommendation 4: Expand stock option eligibility.**  
StockOptionLevel is 0.53 for leavers vs 0.85 for stayers. → Extend equity to Level 2 employees with tenure-based vesting.
5. **Recommendation 5: Deploy the model as an early-warning tool.**  
Recall 0.66 with no overfit. → Integrate into HR dashboards; flag top-decile risk employees quarterly for retention outreach.

### 17.2 Future Work

1. Collect longitudinal data — track employees over multiple years.
2. Add external features — labour market trends, competitor pay data.
3. Try deep learning — with a much larger dataset.
4. Validate retention actions with A/B testing before full rollout.
5. Study causality — use causal inference methods, not just correlation.
6. Balance the minority class — collect more leaver records or apply SMOTE.

## 18. REFERENCES

1. India Data Research Academy. (2026). *Data Science and AI Training Program — Capstone Project Guide*. IDRA.
2. India Data Research Academy. (2026). *Employee Attrition and Workforce Analytics Dataset* (Project #7). Provided via IDRA LMS.

## APPENDIX A. CODE

### A.1 Imports & Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split, StratifiedKFold, cross_val_score,
GridSearchCV
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, confusion_matrix, classification_report,
                             roc_auc_score)

pd.set_option('display.max_columns', None)
sns.set_style('whitegrid')
```

### A.2 Load & Inspect

```
df = pd.read_csv('P_7_WA_Fn-UseC_-HR-Employee-Attrition.csv')

print("Shape:", df.shape)
print("\nFirst 5 rows:")
print(df.head())

print("\nData types:")
print(df.dtypes.value_counts())

print("\nMissing values:", df.isnull().sum().sum())
print("Duplicates:", df.duplicated().sum())

print("\nTarget distribution:")
print(df['Attrition'].value_counts())
print(df['Attrition'].value_counts(normalize=True) * 100)
```

### A.3 Target Analysis

```
plt.figure(figsize=(5, 4))
sns.countplot(x='Attrition', data=df, hue='Attrition', palette='Set1', legend=False)
plt.title('Attrition Distribution')
plt.show()
```

### A.4 Drop Useless Columns & Encode Target

```
df.drop(['EmployeeCount', 'Over18', 'StandardHours', 'EmployeeNumber'],
        axis=1, inplace=True)

df['Attrition'] = df['Attrition'].map({'Yes': 1, 'No': 0})

print("Shape after dropping:", df.shape)
print("Target dtype:", df['Attrition'].dtype)
```

## A.5 Categorical EDA

```
cat_cols = ['OverTime', 'Department', 'JobRole', 'MaritalStatus',
            'BusinessTravel', 'Gender', 'EducationField']

for col in cat_cols:
    rate = df.groupby(col)['Attrition'].mean().sort_values(ascending=False) * 100

    plt.figure(figsize=(9, 4))
    sns.barplot(x=rate.index, y=rate.values, hue=rate.index, palette='Set2', legend=False)
    plt.title(f'Attrition Rate (%) by {col}')
    plt.ylabel('Attrition Rate (%)')
    plt.xticks(rotation=45, ha='right')
    plt.axhline(y=16.12, color='red', linestyle='--', label='Overall rate')
    plt.legend()
    plt.tight_layout()
    plt.show()

    print(f"\n{col}:")
    print(rate.round(2))
```

## A.6 Numerical EDA — Leavers vs Stayers

```
num_cols = ['Age', 'MonthlyIncome', 'YearsAtCompany', 'TotalWorkingYears',
            'YearsInCurrentRole', 'YearsWithCurrManager', 'YearsSinceLastPromotion',
            'DistanceFromHome', 'JobLevel', 'StockOptionLevel',
            'JobSatisfaction', 'EnvironmentSatisfaction', 'WorkLifeBalance',
            'JobInvolvement', 'NumCompaniesWorked', 'TrainingTimesLastYear']

comp = df.groupby('Attrition')[num_cols].mean().T
comp.columns = ['Stayed', 'Left']
comp['Diff_%'] = (comp['Left'] - comp['Stayed']) / comp['Stayed'] * 100).round(2)
print("Mean Comparison (Stayed vs Left):")
print(comp.round(2))

top_num = ['Age', 'MonthlyIncome', 'YearsAtCompany', 'TotalWorkingYears',
            'JobLevel', 'StockOptionLevel', 'YearsInCurrentRole', 'DistanceFromHome']

fig, axes = plt.subplots(2, 4, figsize=(16, 8))
for ax, col in zip(axes.flatten(), top_num):
    sns.boxplot(x='Attrition', y=col, data=df, ax=ax, hue='Attrition',
                palette='Set2', legend=False)
    ax.set_title(col)
    ax.set_xlabel('')
plt.tight_layout()
plt.show()
```

## A.7 Correlation Analysis

```
plt.figure(figsize=(16, 12))
corr = df.corr(numeric_only=True)
mask = np.triu(np.ones_like(corr, dtype=bool), k=1)
sns.heatmap(corr, mask=mask, annot=True, fmt='.2f', cmap='coolwarm',
            center=0, square=True, linewidths=0.5, cbar_kws={'shrink': 0.7})
plt.title('Correlation Heatmap (lower triangle)')
plt.tight_layout()
plt.show()

print("\nTop correlations with Attrition:")
print(df.corr(numeric_only=True)['Attrition'].drop('Attrition').sort_values().round(3))

print("\nHighly correlated feature pairs (|r| > 0.6):")
corr_matrix = df.corr(numeric_only=True).abs()
upper = corr_matrix.where(np.triu(np.ones(corr_matrix.shape), k=1).astype(bool))
```

```
high_corr = [(col, row, upper.loc[row, col])
              for col in upper.columns
              for row in upper.index
              if upper.loc[row, col] > 0.6]
for a, b, r in high_corr:
    print(f" {a} ↔ {b}: {r:.3f}")
```

## A.8 Feature Engineering

```
df['TenureRatio'] = df['YearsAtCompany'] / (df['TotalWorkingYears'] + 1)
df['PromotionLagRatio'] = df['YearsSinceLastPromotion'] / (df['YearsAtCompany'] + 1)
df['ManagerStability'] = df['YearsWithCurrManager'] / (df['YearsAtCompany'] + 1)
df['RoleStagnation'] = df['YearsInCurrentRole'] / (df['YearsAtCompany'] + 1)
df['AgeAtJoin'] = df['Age'] - df['YearsAtCompany']
df['IncomePerLevel'] = df['MonthlyIncome'] / df['JobLevel']
df['HopRate'] = df['NumCompaniesWorked'] / (df['TotalWorkingYears'] + 1)
df['SatisfactionComposite'] = (df['JobSatisfaction'] + df['EnvironmentSatisfaction'] +
                               df['RelationshipSatisfaction'] + df['WorkLifeBalance']) /
4

new_feats = ['TenureRatio', 'PromotionLagRatio', 'ManagerStability', 'RoleStagnation',
             'AgeAtJoin', 'IncomePerLevel', 'HopRate', 'SatisfactionComposite']

print("Correlation of new features with Attrition:")
print(df[new_feats +
         ['Attrition']].corr()['Attrition'].drop('Attrition').sort_values().round(3))
print("\nNew shape:", df.shape)
```

## A.9 Drop Useless Feature & Define Feature Sets

```
df.drop('PromotionLagRatio', axis=1, inplace=True)

target = 'Attrition'

drop_for_lr = ['JobLevel', 'YearsInCurrentRole', 'YearsWithCurrManager',
               'PerformanceRating', 'PercentSalaryHike',
               'YearsSinceLastPromotion', 'RoleStagnation', 'ManagerStability']

all_features = [c for c in df.columns if c != target]
lr_features = [c for c in all_features if c not in drop_for_lr]

print(f"Total features: {len(all_features)}")
print(f"LR features after multicollinearity drop: {len(lr_features)}")
```

## A.10 Encode & Split

```
df_lr = df[lr_features + [target]].copy()
df_tree = df[all_features + [target]].copy()

df_lr_enc = pd.get_dummies(df_lr,
                           columns=df_lr.select_dtypes(include='object').columns.tolist(),
                           drop_first=True)
df_tree_enc = pd.get_dummies(df_tree,
                              columns=df_tree.select_dtypes(include='object').columns.tolist(),
                              drop_first=True)

df_lr_enc = df_lr_enc.astype({c: int for c in
                              df_lr_enc.select_dtypes(include='bool').columns})
df_tree_enc = df_tree_enc.astype({c: int for c in
                                   df_tree_enc.select_dtypes(include='bool').columns})

X_lr = df_lr_enc.drop(target, axis=1)
y = df_lr_enc[target]

X_tree = df_tree_enc.drop(target, axis=1)
y_tree = df_tree_enc[target]
```

```
X_train_lr, X_test_lr, y_train, y_test = train_test_split(
    X_lr, y, test_size=0.2, random_state=42, stratify=y)

X_train_tree, X_test_tree, _, _ = train_test_split(
    X_tree, y_tree, test_size=0.2, random_state=42, stratify=y_tree)

print(f"Train shapes: LR={X_train_lr.shape}, Tree={X_train_tree.shape}")
print(f"Test shapes: LR={X_test_lr.shape}, Tree={X_test_tree.shape}")
```

## A.11 Scale Features

```
scaler = StandardScaler()
X_train_lr_scaled = scaler.fit_transform(X_train_lr)
X_test_lr_scaled = scaler.transform(X_test_lr)

print("Scaled LR train shape:", X_train_lr_scaled.shape)
print("Scaled LR test shape:", X_test_lr_scaled.shape)
```

## A.12 Train Baseline Models

```
lr = LogisticRegression(class_weight='balanced', random_state=42, max_iter=2000)
lr.fit(X_train_lr_scaled, y_train)

dt = DecisionTreeClassifier(class_weight='balanced', random_state=42)
dt.fit(X_train_tree, y_train)

rf = RandomForestClassifier(class_weight='balanced', random_state=42)
rf.fit(X_train_tree, y_train)

def evaluate(name, model, X_tr, X_te):
    y_tr_pred = model.predict(X_tr)
    y_te_pred = model.predict(X_te)
    return {
        'Model': name,
        'Train Acc': round(accuracy_score(y_train, y_tr_pred), 4),
        'Test Acc': round(accuracy_score(y_test, y_te_pred), 4),
        'Precision (Left)': round(precision_score(y_test, y_te_pred), 4),
        'Recall (Left)': round(recall_score(y_test, y_te_pred), 4),
        'F1 (Left)': round(f1_score(y_test, y_te_pred), 4),
        'ROC-AUC': round(roc_auc_score(y_test, model.predict_proba(X_te)[: , 1]), 4)
    }

results = [
    evaluate('Logistic Regression', lr, X_train_lr_scaled, X_test_lr_scaled),
    evaluate('Decision Tree', dt, X_train_tree, X_test_tree),
    evaluate('Random Forest', rf, X_train_tree, X_test_tree)
]

results_df = pd.DataFrame(results)
print(results_df.to_string(index=False))
```

## A.13 Cross-Validation

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

print("5-FOLD CV - F1 on 'Left' class")
print("=" * 60)

for name, model, X_data in [('LR', lr, X_train_lr_scaled),
                             ('DT', dt, X_train_tree),
                             ('RF', rf, X_train_tree)]:
    f1_scores = cross_val_score(model, X_data, y_train, cv=cv, scoring='f1')
    print(f"{name}: F1 = {f1_scores.mean():.4f} (+/- {f1_scores.std():.4f})")

print("\n5-FOLD CV - ROC-AUC")
```



```
print("=" * 60)

for name, model, X_data in [('LR', lr, X_train_lr_scaled),
                             ('DT', dt, X_train_tree),
                             ('RF', rf, X_train_tree)]:
    auc_scores = cross_val_score(model, X_data, y_train, cv=cv, scoring='roc_auc')
    print(f"{name}: AUC = {auc_scores.mean():.4f} (+/- {auc_scores.std():.4f})")
```

## A.14 Hyperparameter Tuning

```
# Decision Tree tuning
param_grid_dt = {
    'max_depth': [3, 4, 5, 6, 8],
    'min_samples_split': [5, 10, 20, 30],
    'min_samples_leaf': [5, 10, 15, 20],
    'criterion': ['gini', 'entropy']
}

grid_dt = GridSearchCV(
    DecisionTreeClassifier(class_weight='balanced', random_state=42),
    param_grid_dt, cv=cv, scoring='f1', n_jobs=-1, verbose=1
)
grid_dt.fit(X_train_tree, y_train)
best_dt = grid_dt.best_estimator_
print("Best DT params:", grid_dt.best_params_)

# Random Forest tuning
param_grid_rf = {
    'n_estimators': [100, 200, 300],
    'max_depth': [4, 6, 8, 10],
    'min_samples_split': [5, 10, 20],
    'min_samples_leaf': [5, 10, 15],
    'max_features': ['sqrt', 'log2']
}

grid_rf = GridSearchCV(
    RandomForestClassifier(class_weight='balanced', random_state=42),
    param_grid_rf, cv=cv, scoring='f1', n_jobs=-1, verbose=1
)
grid_rf.fit(X_train_tree, y_train)
best_rf = grid_rf.best_estimator_
print("Best RF params:", grid_rf.best_params_)
```

## A.15 Extended Model Comparison

```
from sklearn.ensemble import GradientBoostingClassifier, AdaBoostClassifier,
VotingClassifier
from sklearn.svm import SVC

gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
                                max_depth=3, random_state=42)
gb.fit(X_train_tree, y_train)

ada = AdaBoostClassifier(n_estimators=200, learning_rate=0.05, random_state=42)
ada.fit(X_train_tree, y_train)

svm = SVC(kernel='rbf', C=1.0, gamma='scale', class_weight='balanced',
           probability=True, random_state=42)
svm.fit(X_train_lr_scaled, y_train)

voting = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(class_weight='balanced', random_state=42,
max_iter=2000)),
        ('rf', RandomForestClassifier(n_estimators=100, max_depth=4, min_samples_leaf=5,
min_samples_split=20, max_features='log2',
```

```

        class_weight='balanced', random_state=42)),
        ('gb', GradientBoostingClassifier(n_estimators=200, learning_rate=0.05,
        max_depth=3, random_state=42))
    ],
    voting='soft'
)
voting.fit(X_train_lr_scaled, y_train)

extended_models = [
    ('Gradient Boosting', gb, X_train_tree, X_test_tree),
    ('AdaBoost', ada, X_train_tree, X_test_tree),
    ('SVM (RBF)', svm, X_train_lr_scaled, X_test_lr_scaled),
    ('Voting (LR+RF+GB)', voting, X_train_lr_scaled, X_test_lr_scaled)
]

print(f"{'Model':<22} {'Train':>7} {'Test':>7} {'Prec':>7} {'Recall':>7} {'F1':>7}
{'AUC':>7}")
print("-" * 70)

for name, model, X_tr, X_te in extended_models:
    y_tr_p = model.predict(X_tr)
    y_te_p = model.predict(X_te)
    y_te_prob = model.predict_proba(X_te)[:, 1]
    print(f"{name:<22} "
          f"{accuracy_score(y_train, y_tr_p):>7.4f} "
          f"{accuracy_score(y_test, y_te_p):>7.4f} "
          f"{precision_score(y_test, y_te_p):>7.4f} "
          f"{recall_score(y_test, y_te_p):>7.4f} "
          f"{f1_score(y_test, y_te_p):>7.4f} "
          f"{roc_auc_score(y_test, y_te_prob):>7.4f}")

```

## A.16 Threshold Tuning (Logistic Regression)

```

y_prob_lr = lr.predict_proba(X_test_lr_scaled)[:, 1]

thresholds = np.arange(0.2, 0.75, 0.05)

print(f"{'Thresh':>8} {'Prec':>8} {'Recall':>8} {'F1':>8} {'Acc':>8}")
print("-" * 48)

for t in thresholds:
    pred_t = (y_prob_lr >= t).astype(int)
    print(f"{t:>8.2f} "
          f"{precision_score(y_test, pred_t):>8.3f} "
          f"{recall_score(y_test, pred_t):>8.3f} "
          f"{f1_score(y_test, pred_t):>8.3f} "
          f"{accuracy_score(y_test, pred_t):>8.3f}")

```

## A.17 Final Model Evaluation (Logistic Regression)

```

y_final_pred = (y_prob_lr >= 0.5).astype(int)

cm = confusion_matrix(y_test, y_final_pred)
print("Confusion Matrix:")
print(f"
    Predicted Stay    Predicted Leave
Actual Stay      {cm[0,0]:>5}      {cm[0,1]:>5}")
print(f"Actual Leave    {cm[1,0]:>5}      {cm[1,1]:>5}")

print("\nClassification Report:")
print(classification_report(y_test, y_final_pred, target_names=['Stayed', 'Left']))

print("Train Accuracy:", round(lr.score(X_train_lr_scaled, y_train), 4))
print("Test Accuracy :", round(lr.score(X_test_lr_scaled, y_test), 4))
print("ROC-AUC      :", round(roc_auc_score(y_test, y_prob_lr), 4))

plt.figure(figsize=(5, 4))

```

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Stayed', 'Left'], yticklabels=['Stayed', 'Left'])
plt.title('Confusion Matrix - Logistic Regression')
plt.ylabel('Actual')
plt.xlabel('Predicted')
plt.show()
```

## APPENDIX B. ADDITIONAL FIGURES AND TABLES

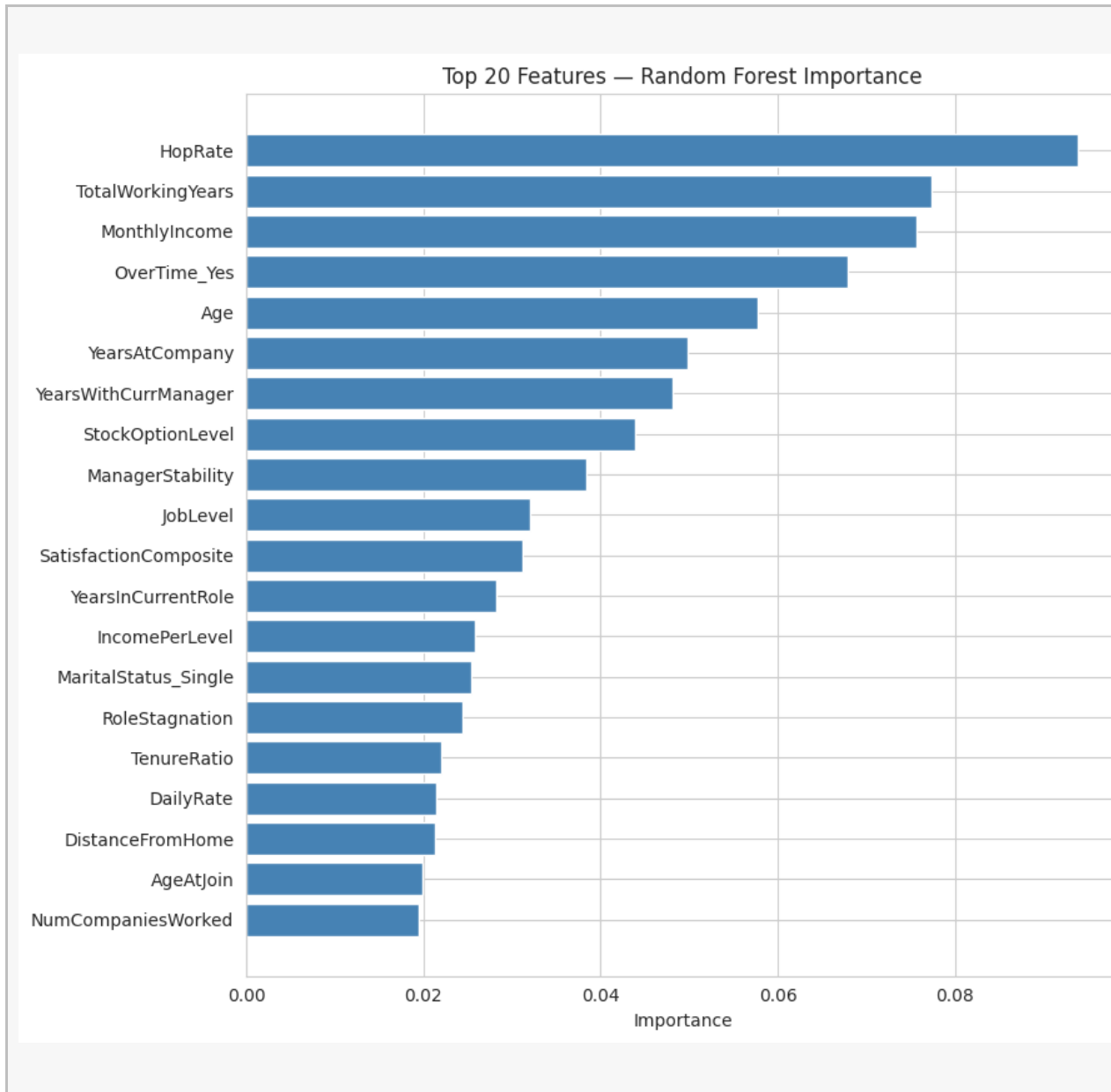


Figure 6. Top 20 Features - Random Forest Importance

Random Forest ranked features by their contribution to prediction. The top five: HopRate (0.094), TotalWorkingYears (0.077), MonthlyIncome (0.076), OverTime\_Yes (0.068), Age (0.058).

**Table 8. Cross-Validation Results (5-Fold)**

| Model                 | Mean F1 | Std     |
|-----------------------|---------|---------|
| Logistic Regression   | 0.5044  | ±0.0222 |
| Random Forest (tuned) | 0.5209  | ±0.0357 |
| Decision Tree (tuned) | 0.4005  | ±0.0304 |
| Voting (LR+RF+GB)     | 0.5810  | ±0.0400 |
| SVM (RBF)             | 0.5410  | ±0.0530 |
| Gradient Boosting     | 0.4877  | ±0.0670 |
| AdaBoost              | 0.1432  | ±0.0770 |

**Table 9. Threshold Tuning — Logistic Regression**

| Threshold | Precision | Recall | F1    | Accuracy |
|-----------|-----------|--------|-------|----------|
| 0.30      | 0.283     | 0.830  | 0.422 | 0.636    |
| 0.40      | 0.340     | 0.745  | 0.467 | 0.728    |
| 0.50      | 0.397     | 0.660  | 0.496 | 0.786    |
| 0.55      | 0.412     | 0.596  | 0.487 | 0.799    |
| 0.60      | 0.439     | 0.532  | 0.481 | 0.816    |

Threshold 0.50 gave the best F1 — the default was already optimal. Lower thresholds increased recall but caused too many false alarms; higher thresholds improved precision but missed too many leavers.